Micro-Partitions and Data Clustering

SNOWFLAKE ARCHITECTURE

Emily Melhuish
Technical Curriculum Developer,
Snowflake

datacamp

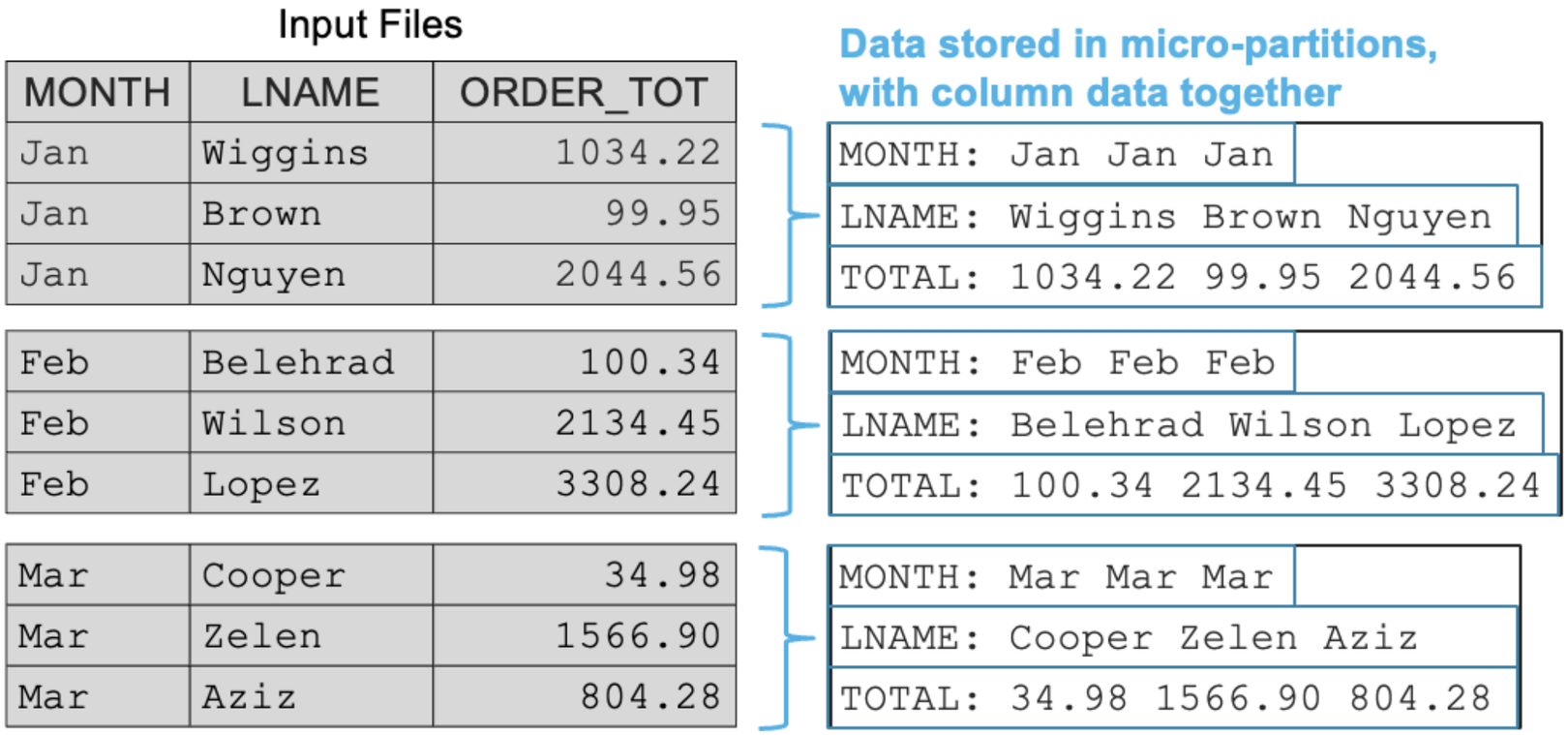
Querying a Large Table

Example: **Subscription**

USER_ID	REGISTRATION_DATE	USER_NAME
912930	2026-04-01	JANEDOE
912931	2026-04-06	JOHNDOE
912932	2026-04-12	JIMMYDOE
...	...	...

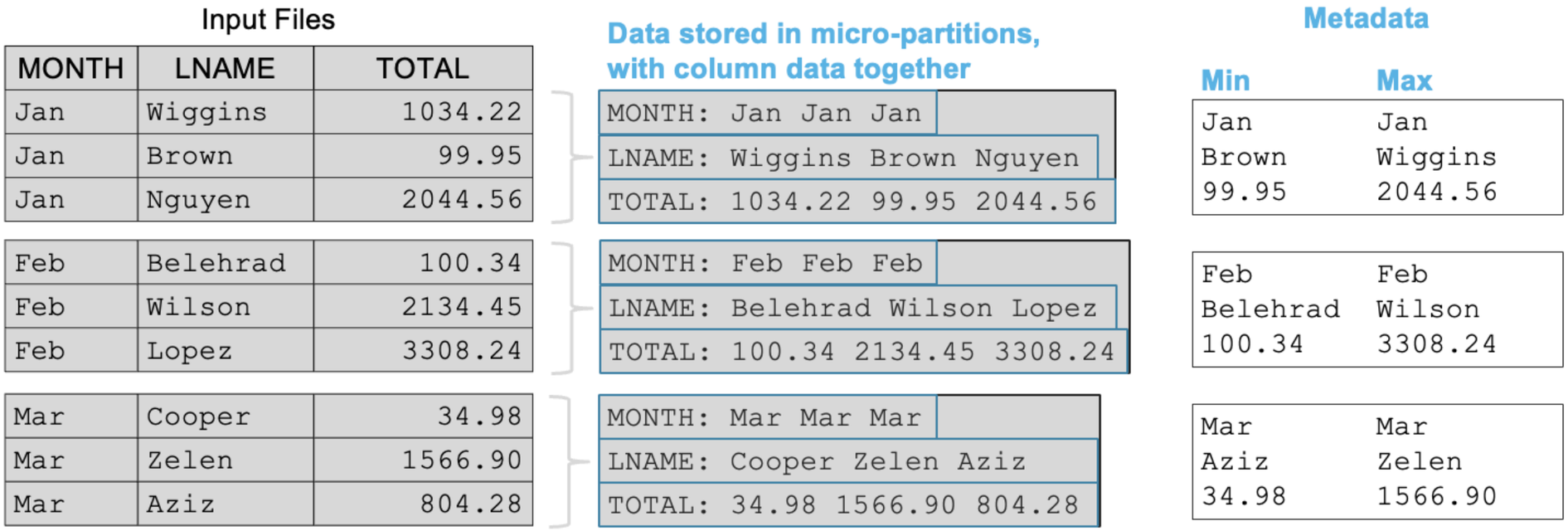
How much data does the query need to read from this table?

Micro-Partitions



- Within micro-partitions columns are stored and compressed separately
- Snowflake only reads columns the query needs

Metadata



- Snowflake stores metadata for each micro-partition
- Stored in the cloud services layer

Partition Pruning

```
SELECT * FROM sales WHERE month = 'May';
```

Min	Max
Jan	Jan
Brown	Wiggins
99.95	2044.56

Min	Max
Apr	Apr
Clark	Patel
103.22	2204.33

Min	Max
Jul	Jul
Adams	Wang
1103.22	21990.23

Min	Max
Oct	Oct
Adamos	Yamamoto
35.12	10335.22

Feb	Feb
Belehrad	Wilson
100.34	3308.24

May	May
Bigliani	Tessay
99.25	14002.34

Aug	Aug
Byrne	Visser
104.45	2210.34

Nov	Nov
Barac	Shevchenko
228.34	1992.45

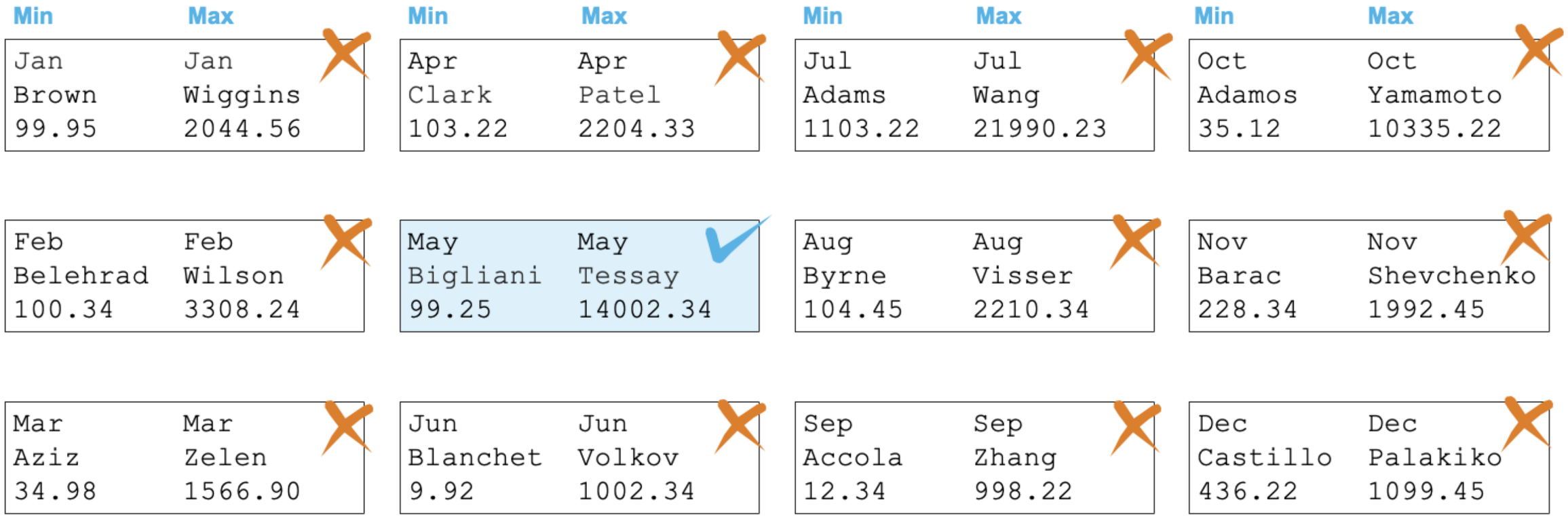
Mar	Mar
Aziz	Zelen
34.98	1566.90

Jun	Jun
Blanchet	Volkov
9.92	1002.34

Sep	Sep
Accola	Zhang
12.34	998.22

Dec	Dec
Castillo	Palakiko
436.22	1099.45

Partition Pruning



- Snowflake skips micropartitions that can't possibly contain the data your query needs.
- It only reads the relevant slices = partition pruning

Clustering

Input Files

MONTH	LNAME	ORDER_TOT
Jan	Wiggins	1034.22
Jan	Brown	99.95
Jan	Nguyen	2044.56
Feb	Belehrad	100.34
Feb	Wilson	2134.45
Feb	Lopez	3308.24
Mar	Cooper	34.98
Mar	Zelen	1566.90
Mar	Aziz	804.28

Data stored in micro-partitions, with column data together

```
MONTH: Jan Jan Jan
LNAME: Wiggins Brown Nguyen
TOTAL: 1034.22 99.95 2044.56

MONTH: Feb Feb Feb
LNAME: Belehrad Wilson Lopez
TOTAL: 100.34 2134.45 3308.24

MONTH: Mar Mar Mar
LNAME: Cooper Zelen Aziz
TOTAL: 34.98 1566.90 804.28
```

- Date-based queries often prune well (for example, time series)
- Pruning is not useful for values that are scattered across partitions (for example, region)
- **Clustering key:** Snowflake organizes micro-partitions around that column

Clustering Key: When to Use

Checklist

- Large tables (100GB+)
- Consistently filtering on the same column
- Poorly clustered column

Clustering Key: Cardinality and Cost

Cardinality

sweet spot — not too few, not too many

Column	Distinct values	Good?
region	5–20	**Yes**
country	50–200	**Yes**
customer_id	millions	**No**
is_active	2 (boolean)	**No**

too low ⟵ sweet spot ⟶ too high

Cost

automatic reclustering uses serverless compute

New data arrives
clustering depth degrades

↓

Snowflake reclusters
serverless — not your warehouse

↓

Added to your bill
ongoing background credit usage

Only cluster when:

query savings > reclustering cost
large table + frequent filtered queries

Let's practice!

SNOWFLAKE ARCHITECTURE

Table Types and Views

SNOWFLAKE ARCHITECTURE

Emily Melhuish
Technical Curriculum Developer,
Snowflake

datacamp

Table Types and Views

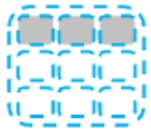
SNOWFLAKE TABLE TYPES

Permanent

Transient*

Temporary

VIEW TYPES

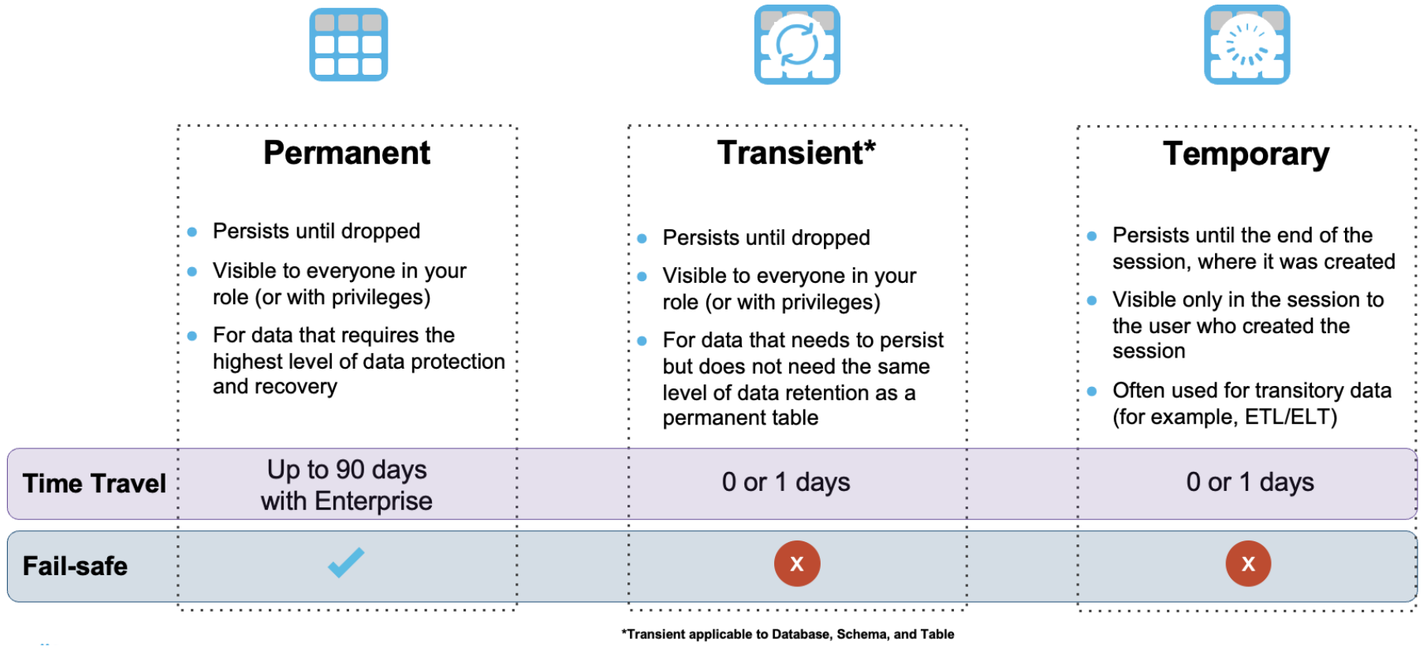


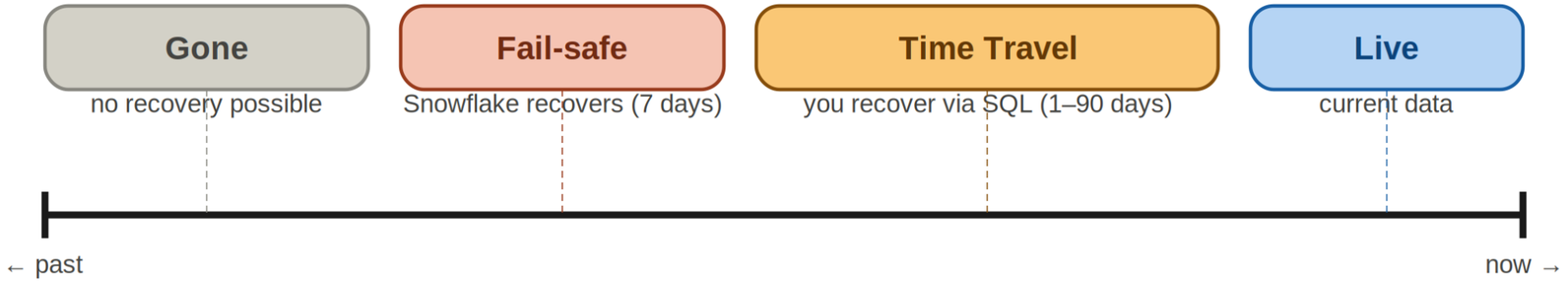
View

Materialized View

Secure your views 

Permanent, Transient and Temporary





Permanent, Transient and Temporary

Permanent

default table type

persists until dropped

full data protection

visible to role members

Transient

staging & intermediate

persists until dropped

reduced protection

visible to role members

Temporary

session only

vanishes on logout

no protection

visible to creator only

Dynamic, External, and Hybrid

Dynamic

- Materializes results of a query
- Privileges assigned to create and/or work with dynamic tables
- Table automatically kept up to date based on user-specified lag time

External

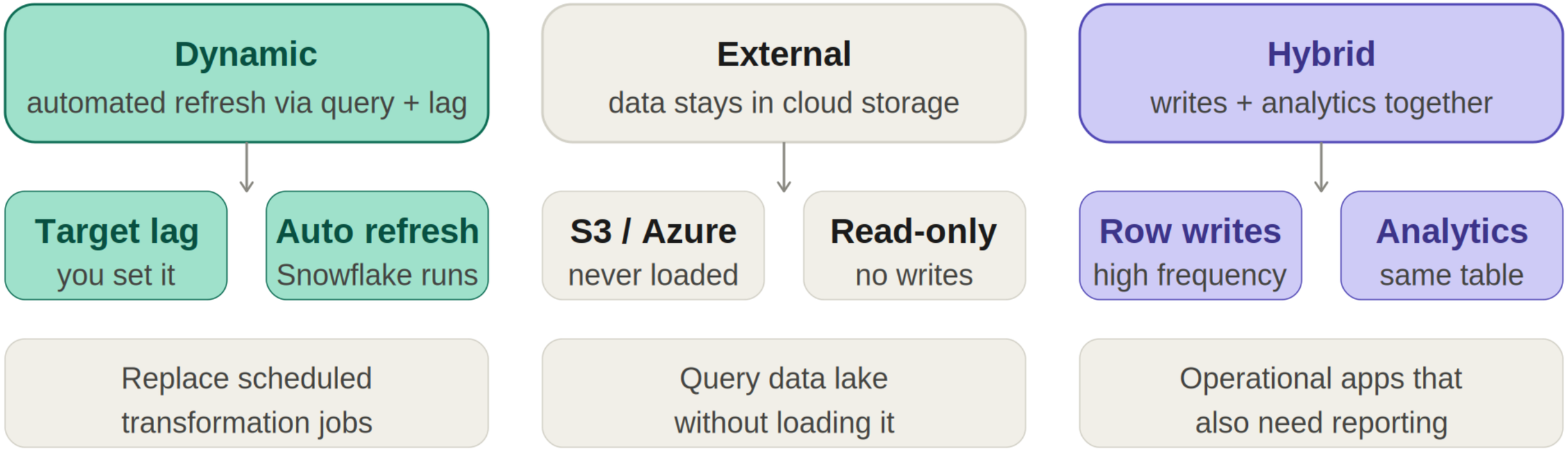
- Persists until dropped
- Visible to everyone in your role (or with privileges)
- Data accessed via an external stage
- Read-only

Hybrid

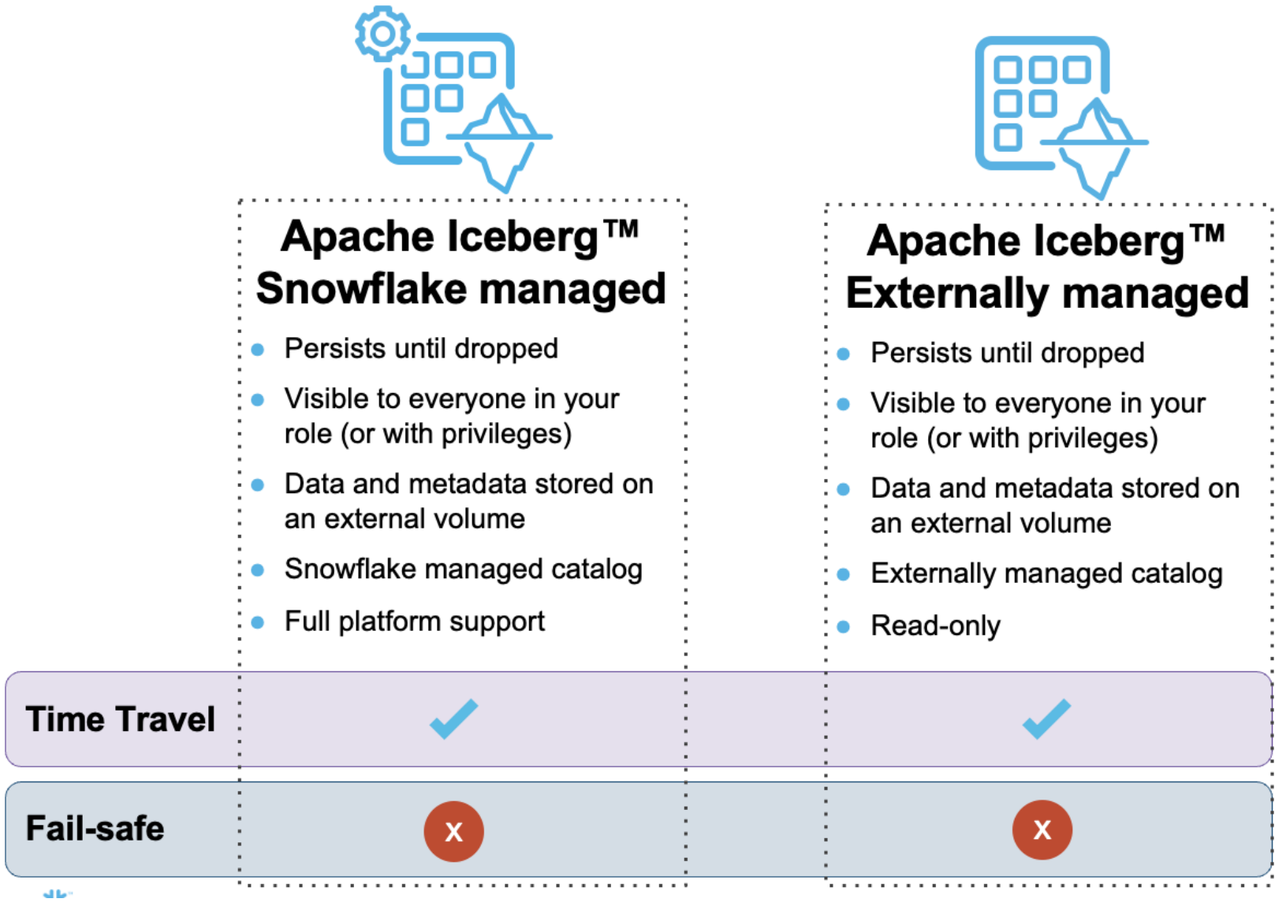
- Persists until dropped
- Visible to everyone in your role (or with privileges)
- Optimized for hybrid transactional and operational workloads

	Dynamic	External	Hybrid
Time Travel	✓	X	✓
Fail-safe	✓	X	X

Dynamic, External, and Hybrid



Apache Iceberg™



Apache Iceberg™

Snowflake managed
Snowflake owns the catalog + metadata

Catalog
inside Snowflake

Read + Write
full platform support

Best when Snowflake is your primary engine

Externally managed
catalog lives outside Snowflake

Catalog
AWS Glue etc.

Read-only
from Snowflake

Best when data lake is the source of truth across engines

Time Travel and Fail-safe (with table types)

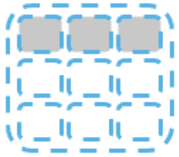
Time Travel

- Way to recover data

Fail-safe

- Separate, short safety net after that window

Standard View and Materialized View



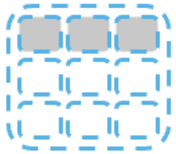
View

- Default view type
- Named definition of a query
- Results are generated at run time

Materialized View

- Results generated at creation time
- Stores results using micro-partitions
- Auto-refreshed

Secure View



View

- Default view type
- Named definition of a query
- Results are generated at run time

Materialized View

- Results generated at creation time
- Stores results using micro-partitions
- Auto-refreshed

Secure your views 

- By default, View DDL is visible to any role with access
- Views and materialized views both support a secure version
- Secure view definition and details only visible to authorized users
- Snowflake query optimizer bypasses some of the optimizations used for regular views

Let's practice!

SNOWFLAKE ARCHITECTURE

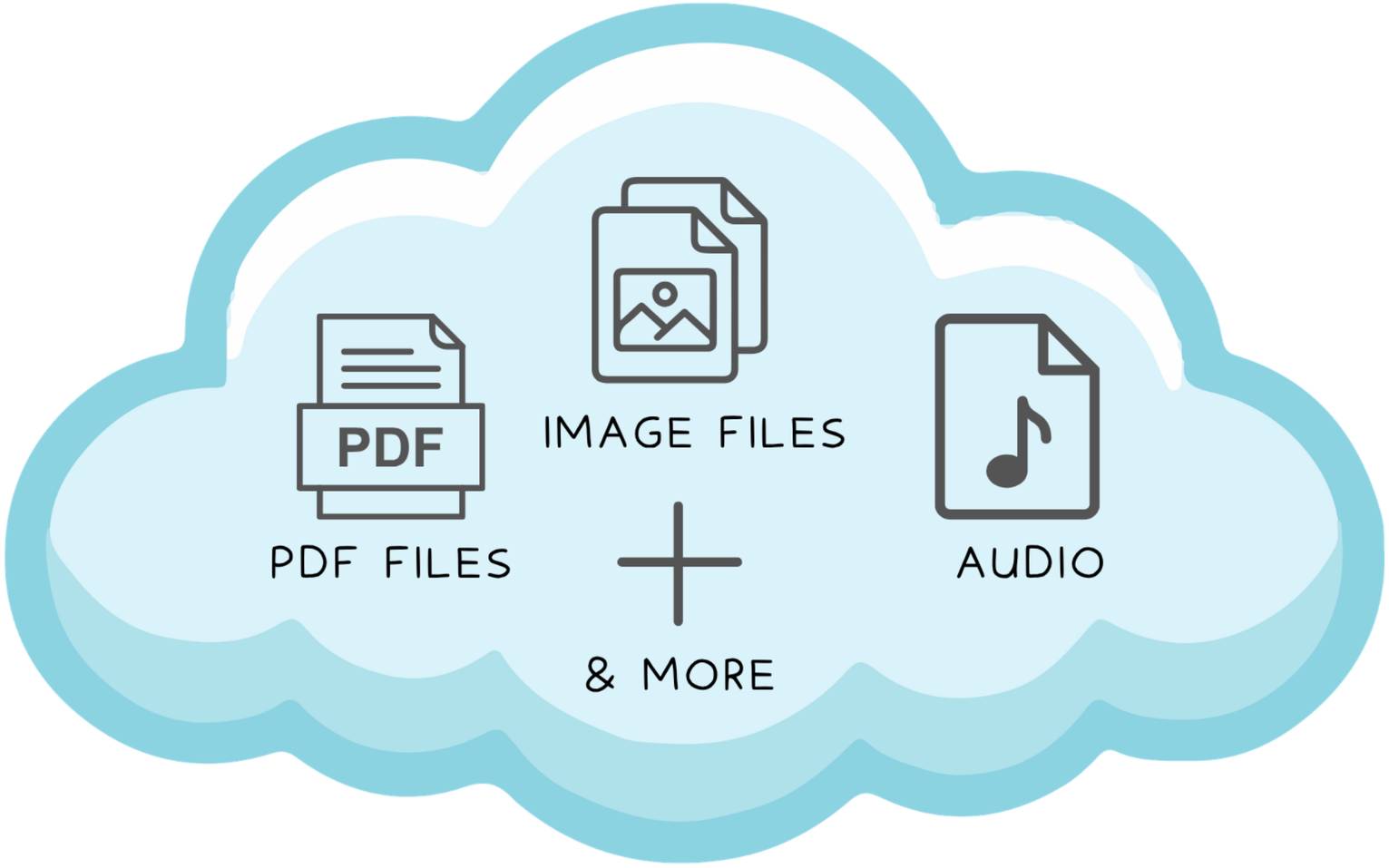
Working with Unstructured Data

SNOWFLAKE ARCHITECTURE

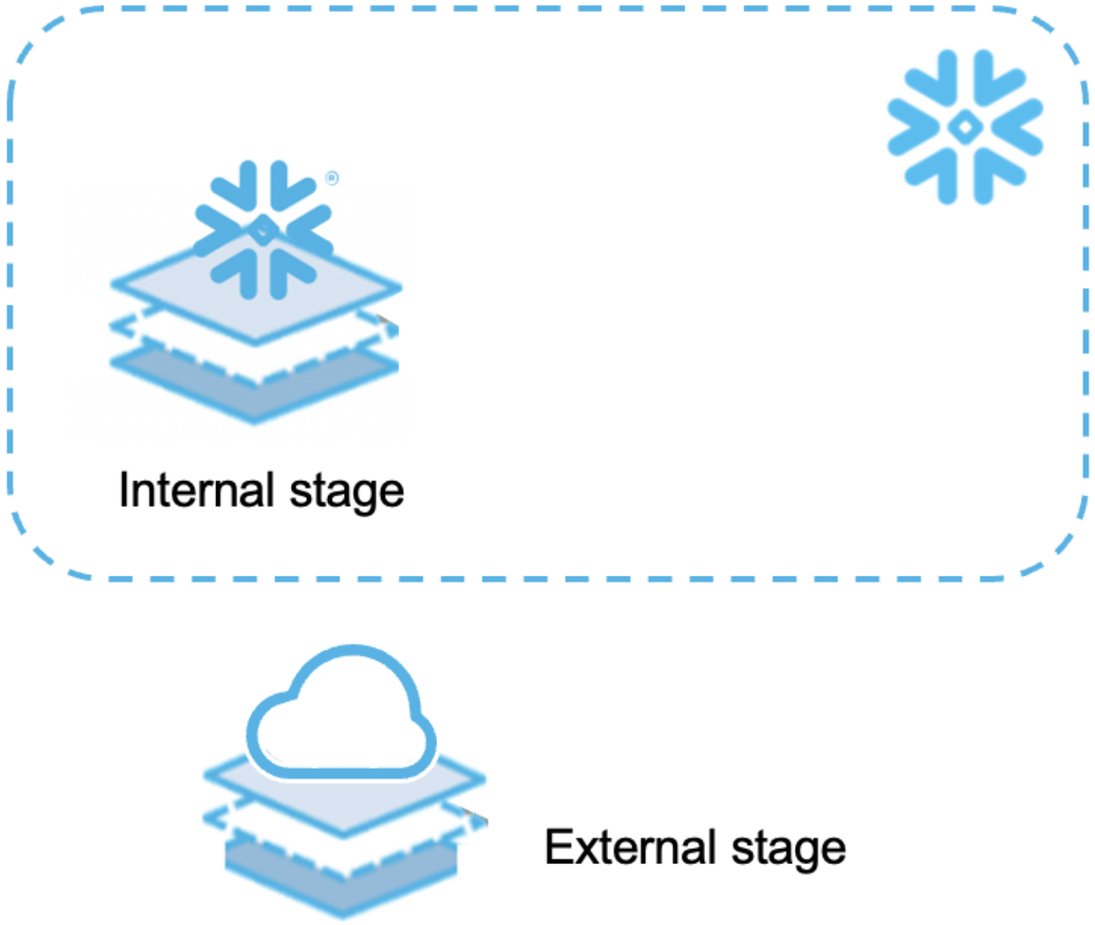
Emily Melhuish
Technical Curriculum Developer,
Snowflake

datacamp

Beyond Tables

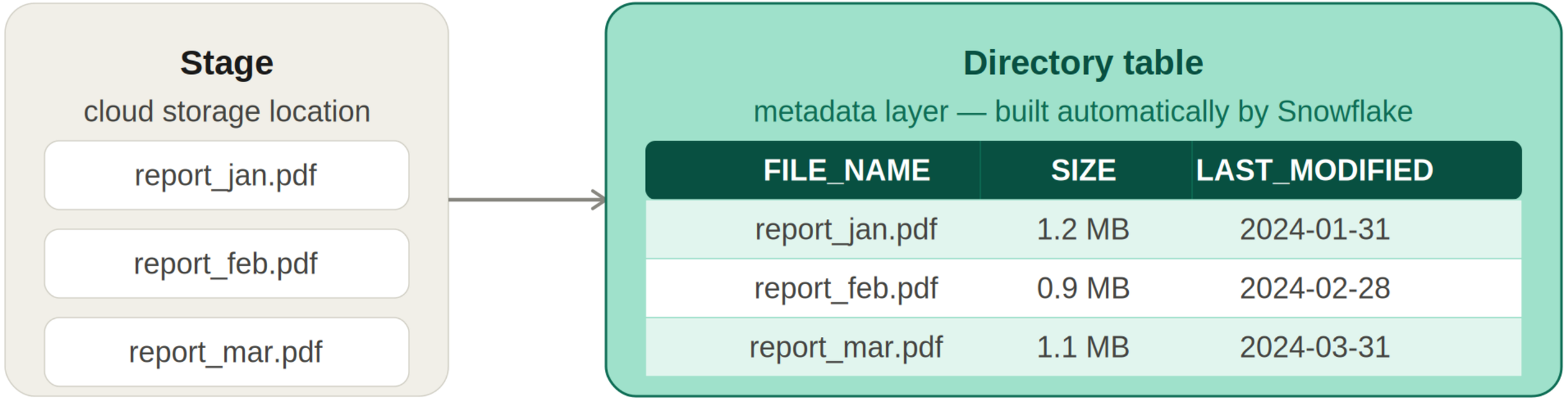


Stages: The Landing Zone for Files



- Internal: Managed by Snowflake
- External: Files in the Cloud
- unstructured data lives in a stage

Directory Tables



Directory Tables Syntax

ENABLE DIRECTORY

```
CREATE STAGE snowy_peak_files_stage
  DIRECTORY = (ENABLE = TRUE);
```

Refresh directory metadata (`ALTER STAGE`)

```
ALTER STAGE snowy_peak_files_stage REFRESH;
```

Directory Tables

Stage

cloud storage location

- report_jan.pdf
- report_feb.pdf
- report_mar.pdf

Directory table

metadata layer — built automatically by Snowflake

FILE_NAME	SIZE	LAST_MODIFIED
report_jan.pdf	1.2 MB	2024-01-31
report_feb.pdf	0.9 MB	2024-02-28
report_mar.pdf	1.1 MB	2024-03-31

Manual refresh

you trigger it on demand

best for testing

and ad hoc loads

Automatic refresh

Snowflake triggers on file arrival

best for production pipelines

requires cloud event notifications

Querying a Directory Table

SQL

```
SELECT
file_name
, size
, last_modified
FROM DIRECTORY(@snowy_peak_files_stage);
```

- Audit files in the stage
- Identify stale files
- Feed downstream processes

Pre-signed URLs

- External partners and applications cannot access a stage
- Snowflake can expose URL or credentials to grant partners access: Stage URLs, scoped file URLs, pre-signed URLs and related helpers

SQL

```sql
SELECT GET_PRESIGNED_URL(@snowy_peak_files_stage, 'avalanche_forecast.pdf', 3600);
```

- You pass stage, file path and lifetime in seconds
- Useful for partners who do not have Snowflake logins

Let's practice!

SNOWFLAKE ARCHITECTURE